

A Sandboxed, Conversational, Multi-Backend System for Low-Resource Text Classification and LLM-Synthesized Reinforcement-Learning Environments

Anonymous submission. All experimental numbers reported in this paper were produced by the accompanying codebase and are reproducible via the scripts in eval/; raw outputs are stored under outputs/.

Abstract

We describe a working system that lets a user specify, train, and deploy machine learning models entirely through natural-language dialogue, across two paradigms: low-resource text classification (10–200 labeled examples per class) and reinforcement learning (RL) with environments synthesized on the fly by a large language model (LLM) from a natural-language task description. The system’s central engineering contribution is a safety pattern for executing LLM-generated, Turing-complete code (PyTorch classification heads and Gymnasium-style RL environments) without full sandboxing guarantees: static AST allow-listing, a second layer of restricted-builtin code execution, and a wall-clock-bounded subprocess as the final backstop, composed with a three-tier fallback ladder (LLM-generated classifier $\rightarrow$ fine-tuned pretrained backbone $\rightarrow$ classical sklearn baseline) that degrades gracefully when LLM code generation fails. We report four real, reproducible experiments rather than claims: (1) the system’s zero-configuration sklearn backend is statistically indistinguishable from hand-tuned classical baselines on four real datasets (Wilcoxon signed-rank, all $p > 0.05$); (2) a red-team suite of 45 malicious and benign code samples against the static sandbox layer; (3) an empirical characterization of how often LLM-generated classifier code succeeds before the fallback ladder degrades it; and (4) a study of whether LLM-synthesized RL environments, validated only by static analysis, actually train to non-trivial reward improvement across a diverse set of natural-language task descriptions – where a static sandbox pass rate of 81% drops to a 31% end-to-end completion rate, and direct diagnosis traces most of that gap to a single, specific, actionable cause: generated environments routing stochastic transitions through unseeded randomness instead of Gymnasium’s own seeded RNG, which

its `check_env` correctness checker (not our sandbox) correctly rejects. We report failures as well as successes, and we are explicit about what this evaluation does and does not establish.

1 Introduction

Making machine learning accessible to non-experts is a long-standing goal of the AutoML literature. Existing systems (Section 2) largely automate a fixed, hand-engineered search space over models and hyperparameters. A separate line of recent work uses large language models as agents that write and execute code to accomplish tasks, raising a distinct engineering problem: if the system lets an LLM generate the actual training code (not just configuration), how much of that code can be executed safely, and what happens when the generated code is wrong or unsafe?

This paper describes a system built to explore that question concretely, across two settings: (a) low-resource text classification, where the system can optionally let an LLM design a custom PyTorch classification head, and (b) reinforcement learning, where the LLM must synthesize the entire training environment (state, action space, reward function, and termination condition) from a natural-language description, since no other component of the pipeline can supply this.

We want to be precise about what this paper claims. This is a systems paper, not a new learning algorithm. We do not claim state-of-the-art results on standard large-scale benchmarks, and we did not run a human-subject study to validate whether natural-language clarification dialogue is an effective substitute for an ML expert configuring the system by hand – that would require an IRB-approved user study, which is out of scope here and is listed explicitly in Section 5. Our contribution is threefold:

1. A concrete, reusable engineering pattern for executing LLM-generated training code (classification heads and RL environments) with three independent, layered defenses, empirically red-teamed rather than asserted safe.
2. A three-tier fallback ladder (custom LLM-generated model $\rightarrow$ fine-tuned pretrained backbone $\rightarrow$ classical baseline) and an empirical characterization of how often the riskiest, most novel tier actually succeeds before falling back.
3. An empirical study of whether natural-language-specified, LLM-synthesized RL environments actually produce learnable tasks, across a deliberately diverse set of task descriptions that avoid naming well-known benchmark environments (to reduce the chance the LLM is simply reproducing memorized reference code).

2 Related Work

AutoML. Systems such as AutoGluon, auto-sklearn, and AutoML-Zero automate model/hyperparameter search over a fixed space of established methods. Our system automates the same kind of decision (which model family, which hyperparameters) but adds a qualitatively different option – letting an LLM design new model code at request time – which these systems do not attempt, precisely because it reintroduces the code-safety problem that a fixed search space avoids by construction.

Conversational / interactive ML systems. Prior work on interactive model building typically focuses on visual or notebook-based interfaces rather than open-ended natural-language dialogue driving every stage (clarification, data collection, backend selection, training, and reporting) of the pipeline.

LLM code-generation agents and sandboxing. A growing literature studies how to safely execute code written by an LLM agent, typically via containerization, syscall filtering, or restricted interpreters. Our approach is narrower and more specific to this domain: rather than a general sandboxed interpreter, we combine (a) a static AST allow-list scoped tightly to the exact contract the generated code must satisfy (e.g., a class implementing `__init__`/forward for a classifier, or `reset/step` for an RL environment), (b) a restricted-builtin `exec()` environment as a second, independent layer, and (c) a wall-clock-bounded subprocess as the final backstop against anything the first two layers miss (notably, resource-exhaustion attacks that have no fixed syntactic signature – see Section 4.3).

Natural-language-driven RL environment / reward synthesis. Recent work (e.g., LLM-driven reward-function design and skill/curriculum synthesis in embodied agents) shows LLMs can propose reward functions or high-level skills for RL. Our RL setting differs in scope: the LLM must synthesize the entire environment dynamics (state transition and termination logic), not only a reward function layered on top of an existing, trusted environment, which is a strictly larger and less constrained code-generation surface.

3 System Design

3.1 Conversational orchestration

The system exposes a WebSocket-backed multi-turn dialogue. A finite-stage state machine (task clarification $\rightarrow$ data/environment collection $\rightarrow$ backend selection $\rightarrow$ training-configuration $\rightarrow$ training $\rightarrow$ reporting) processes each user message; a stage advances as soon as it is satisfied, so a single message that already contains everything needed (e.g., a fully-specified task description with pasted examples) skips directly to training without being re-asked. Training progress is streamed into the same message channel as ordinary chat messages, so the frontend’s visualization logic is agnostic to whether the underlying backend is a classical classifier, a fine-tuned transformer, or a reinforcement-learning policy.

3.2 Three-tier classification fallback ladder

For text classification, the system attempts, in order: (1) an LLM-generated, sandbox-validated custom PyTorch classification head over TF-IDF features; (2) if that fails validation or training twice, a LoRA-fine-tuned pretrained transformer backbone from a small fixed whitelist; (3) if that also fails, a classical TF-IDF + sklearn classifier (logistic regression / linear SVM / SGD, chosen automatically by dataset size), which is guaranteed to succeed and is never itself sandboxed since it runs no LLM-generated code.

3.3 RL environment synthesis

For reinforcement learning, the LLM is prompted to generate a `gymnasium.Env` subclass matching a fixed contract (`__init__` with no required arguments beyond `self`; `reset(seed, options) -> (obs, info)`; `step(action) -> (obs, reward, terminated, truncated, info)`). The generated class is (a) statically validated (Section 3.4), (b) instantiated in an

isolated subprocess and wrapped unconditionally in a TimeLimit wrapper regardless of whether the generated code sets terminated correctly, (c) checked with Gymnasium’s own `check_env` utility as a free correctness pre-check, and only then (d) trained with Stable-Baselines3’s PPO or DQN (chosen automatically by action-space type) – i.e., the training algorithm is always trusted library code; only the environment is LLM-generated, which deliberately minimizes the code-execution attack surface relative to also letting the LLM generate the policy network.

3.4 Sandbox design

Both the classification-head and RL-environment code paths share the same three-layer defense-in-depth pattern:

1. Static AST gate. Only a small, explicit set of import modules is allowed (`torch/torch.nn/torch.nn.functional` for classifiers; `gymnasium/gymnasium.spaces/numpy/math/random` for RL environments – `torch` is deliberately excluded from the RL allow-list, since the training algorithm supplies the policy network and the environment has no legitimate need for it). A further denylist blocks known dangerous names (`os`, `subprocess`, `socket`, `eval`, `exec`, `open`, `__import__`, ...) and classic sandbox-escape dunders (`__globals__`, `__subclasses__`, `__reduce__`, ...), and rejects any decorator use and any structural deviation from the required class contract.
2. Restricted `exec()` builtins. Even code that passed the static gate is executed with a minimal builtins dictionary (roughly 25 names – `range`, `len`, `isinstance`, etc. – notably excluding `type`), so most static-analysis bypasses that produce syntactically clean but semantically unexpected code still fail at execution time.
3. Wall-clock-bounded subprocess. All training runs in a multiprocessing subprocess (spawn context) with `join(timeout)` followed by escalating `terminate()/kill()` if the process is still alive. This is the only layer that can stop resource-exhaustion or infinite-loop attacks, which have no fixed syntactic signature for the first two layers to catch.

During development of this system we discovered and fixed a genuine deadlock in this subprocess layer: the original implementation called `Process.join()` before draining the result queue, which deadlocks once

the pickled result (e.g., serialized RL policy weights) exceeds the OS pipe buffer – the child’s queue-feeder thread blocks on `write()` waiting for a reader that will never arrive, while the parent blocks in `join()` waiting for a child that cannot exit. We mention this because it is exactly the kind of subtle correctness bug that empirical testing (Section 4.2, 4.4) surfaces and code review alone would not have.

4 Experiments

All experiments were run on a single Apple Silicon machine (MPS, no CUDA, no 4/8-bit quantization support); wall-clock numbers should be read as characterizing this specific hardware, not general system throughput. The local LLM used throughout is Ollama’s `qwen3.6:35b-a3b` (no API key, no external cost), except where noted.

4.1 Experiment 1: does the automatic sklearn backend match manually-chosen classical baselines?

We compare the system’s zero-configuration sklearn backend (Ours-Base: automatic model-family selection by dataset size; Ours-SMOTE: the same, plus automatic SMOTE oversampling) against manually-specified classical baselines (TF-IDF + logistic regression, TF-IDF + linear SVM) on four datasets: 20 Newsgroups (4-class and binary subsets), a bundled 30-example customer-service-ticket dataset (5 classes), and `sepidmnorozy/Chinese_sentiment` (a real, third-party Hugging Face dataset, included to cover the system’s bilingual Chinese/English target use case). We sweep 10/20/50/100 examples per class and repeat every configuration 10 times, resampling which specific examples are drawn each repeat (`seed = 42 + repeat`) to obtain genuine variance – the underlying classifier’s own `random_state` is fixed at 42 by the evaluated system itself, a real property of the implementation we did not alter and report as a limitation (Section 5), rather than paper over it.

We discovered and fixed two bugs in the (previously unrun) evaluation harness before trusting its output: repeats had been drawing the identical subsample every time (making reported variance always exactly zero), and the Markdown-table export function called itself recursively with no base case, crashing with a `RecursionError` on its first real invocation. Both are described in Section 5 and fixed in the accompanying code.

Result. Table 1 shows F1 (weighted) for Ours-SMOTE vs. the two baselines. A paired Wilcoxon

Table 1: F1 (weighted), mean over 10 repeats, at $n=100$ examples/class. Full sweep (10/20/50/100) and per-repeat std. in outputs/exp1_sklearn_baselines/.

System	20NG-4	20NG-2	zh-sent.	tickets
LogReg	0.7028	0.8800	0.7320	0.0083
SVM	0.7243	0.8890	0.7323	0.0083
Ours-Base	0.7145	0.8836	0.7094	0.0152
Ours-SMOTE	0.7160	0.8840	0.7074	0.0291

signed-rank test across all 16 (dataset $\times$ sample-size) configurations shows no statistically significant difference in any comparison (Ours-SMOTE vs. LogReg: $p = 0.379$; vs. SVM: $p = 0.178$; Ours-Base vs. LogReg: $p = 0.097$; all $n = 16$). We report this honestly as a parity finding rather than an improvement: the system matches hand-tuned classical baselines while requiring no manual model or hyperparameter choice from the user, which is the claim we can actually support – not that automatic selection or SMOTE improves raw accuracy.

We also note that the bundled 30-example ticket dataset produces near-floor F1 (0.008–0.029) for every evaluated system, including the plain baselines, and identical results across all requested sample sizes (because the dataset has only ~ 6 examples per class in total, well below what any of the four requested sample-size settings could actually supply). We report this as a genuine limit of the few-shot regime this system targets, not a bug specific to our method, since the plain sklearn baselines degrade identically.

4.2 Experiment 2: does the fallback ladder help, and how often does the LLM-generated tier actually work?

We compare `model_backend=sklearn` against `model_backend=custom_nn` (which triggers the full ladder: an LLM-generated classification head, falling back to a fine-tuned pretrained backbone, falling back to sklearn, if earlier tiers fail validation or training) across 3 seeds on a real dataset at $n=20$ /class.

Methodology deviation, disclosed. The plan called for the bundled customer-ticket dataset, since it best matches this system’s target use case; Experiment 4.1 showed that dataset is too small (6 examples/class) to support a genuine $n=20$ /class draw and produces degenerate results for every system. We substituted `chinese_sentiment` (Section 4.1), which has enough examples to actually support the requested sample size, after discovering

Table 2: Fallback ladder vs. sklearn-only, 3 seeds, $n=20$ /class, `chinese_sentiment`.

	sklearn-only	fallback ladder
Mean best metric (accuracy)	0.657	0.563
Mean wall-clock (s)	75.4	77.6
<code>custom_nn</code> 1st-try success rate	—	0.667
Reached pretrained/sklearn tier	—	0/3

the problem empirically rather than assuming the original plan would work.

Result. Table 2 shows the outcome (full per-seed detail in outputs/exp2_fallback_ladder/summary.json). The LLM-generated `custom_nn` tier succeeded on its very first attempt in 2 of 3 seeds and needed exactly one in-tier retry (not a full fallback to the pretrained backbone or sklearn) in the third, so all 3 seeds were ultimately served by the LLM-generated classifier tier – the fallback ladder never had to reach its second or third rung in this run. With only 3 seeds, the mean accuracy is not meaningfully different between conditions (0.657 sklearn vs. 0.563 ladder; the ladder’s seed-2 run, at 0.27, is the visible outlier driving this). We report this honestly rather than only reporting favorable seeds: at this sample size we cannot claim the ladder improves accuracy, only that it reaches a working classifier via a qualitatively different route (LLM-generated code rather than a fixed model family) at comparable wall-clock cost, with graceful in-tier retry when the first generation attempt fails.

4.3 Experiment 3: static sandbox red-team evaluation

We construct a red-team suite of malicious and benign code samples against both sandboxes (classification-head and RL-environment), covering nine attack categories: dangerous imports (`os`, `subprocess`, `socket`, `urllib`, `ctypes`), dangerous builtins (`eval`, `exec`, `open`, `getattr`), classic sandbox-escape dunder chains (`__globals__`, `__subclasses__`), decorator-hidden calls, `import-allowlist` bypasses (`importlib.import_module`, `__import__`), and structural violations (missing required methods/attributes, wrong base class, multiple top-level classes used to hide a payload in a helper class). Benign samples span simple/medium/complex legitimate architectures and environments.

Result. Both sandboxes achieve perfect precision, recall, and accuracy on this suite (`nn_sandbox`: 20 malicious + 8 benign, 28/28 correct; `rl_sandbox`:

12 malicious + 5 benign, 17/17 correct; see `outputs/exp3_sandbox_redteam/summary.json` for the full per-category breakdown). We are explicit about what this does and does not show: (a) it evaluates only the static AST layer in isolation, not the composition of all three defense layers, since constructing a working end-to-end exploit through all three was out of scope; (b) resource exhaustion (unbounded loops, unbounded memory growth per step) is not evaluated here because it has no fixed syntactic signature for a static analyzer to key on by design – the actual defense is the wall-clock subprocess timeout, which we verified fires correctly during real training runs in Experiments 4.2 and 4.4, not the sandbox; (c) we separately found and report one genuine static-analysis gap discovered during this work: `class Evil(torch.random.Module)` passes `nn_sandbox.validate()` because the base-class `check` inspects only the terminal attribute name (`"Module"`) via `ast.Attribute.attr`, not the full dotted path or an actual subclass relationship – `torch.random` has no `Module` attribute, so this fails at actual `exec()` time with `AttributeError` rather than executing anything, i.e. it fails closed at a different layer, but it is a real precision gap in the static layer’s stated contract that we did not paper over.

4.4 Experiment 4: does LLM-synthesized RL environment generation actually work?

We hand-write 16 natural-language RL task descriptions spanning navigation, resource scheduling, simple games, and control, deliberately avoiding naming well-known benchmark environments (e.g., describing “a pole balanced on a base that can move left or right” rather than “`CartPole`”), to reduce the chance the LLM is reproducing memorized reference implementations rather than genuinely synthesizing an environment from the description. Each task is run once through the real pipeline (LLM environment synthesis → static validation, with one retry on failure → `Stable-Baselines3` training).

Result. Table 3 summarizes the outcome (full per-task detail, including every failure reason, in `outputs/exp4_rl_env_study/summary.json`). The static sandbox passes the generated environment on the first attempt 81% of the time, but the pipeline only reaches a usable trained policy for 5 of 16 tasks (31%) – most tasks that pass the sandbox still fail later, and we tracked down why rather than leaving it as an unexplained gap.

Root cause, identified by direct diagnosis. We manually re-ran two of the ten failed tasks (“room

Table 3: RL environment synthesis study, 16 natural-language task descriptions.

Metric	Value
Completion rate (usable trained policy)	0.313 (5/16)
Sandbox pass rate (first try)	0.813
Mean wall-clock per task (s)	29.1
Runs with positive reward improvement	2/5 completed

temperature control” and “inventory restocking”) with full event logging. Both failed at the exact same point, with the exact same message, from `Gymnasium`’s own `check_env` correctness checker (not our sandbox): “Deterministic step observations are not equivalent for the same seed and action.” In both cases the LLM had written stochastic dynamics (ambient temperature drift; random daily demand) using Python’s global random module or unseeded numpy calls directly, rather than routing randomness through `self.np_random` – the RNG that `Gymnasium`’s own `Env.reset(seed=...)` sets up specifically so that environment transitions are reproducible given a seed. This is a mechanistic, actionable finding: the environment-design prompt does not currently instruct the LLM to use `self.np_random` for stochastic transitions, and this appears to be the dominant cause of the gap between an 81% sandbox pass rate and a 31% end-to-end completion rate. The one remaining distinct failure mode we observed was the LLM’s JSON response being truncated mid-string for one task (`elevator_dispatch`), consistent with the response hitting the design call’s 1800-token budget for a longer environment.

Among the 5 completed tasks, reward improved from the first to the last training segment in 2 of 5 (mean improvement +4.23 where measured, but this average is dominated by one large positive outlier and should not be read as a typical effect size at $n = 5$).

5 Limitations

We list these explicitly rather than folding them into future work, because several of them bound what can be concluded from Section 4.

- Simulated training curves in the sklearn backend. The sklearn backend’s reported “epoch” is not a gradient-descent epoch – it is an increasing fraction of the training data (30%, 45%, ..., 100%) refit from scratch each step, a legitimate way to visualize a learning curve

for non-iterative classifiers, but its reported `train_loss/val_loss` values are not measured losses: they are computed as `base_loss = 1 - val_metric` plus injected Gaussian noise. We discovered this while preparing this paper and do not report any loss-curve numbers as empirical results anywhere in this paper – only `val_metric` (F1/accuracy, computed by real `predict()` calls against a real held-out split) is used.

- Fixed random seeds inside the evaluated system. Every sklearn classifier and train/validation split inside the evaluated codebase uses a hardcoded `random_state=42`. Our multi-repeat variance in Experiment 4.1 comes from varying which examples are subsampled per repeat, not from varying the classifier’s own internal randomness, which we did not alter in the evaluated system.
- Small numbers of seeds for the expensive experiments. Experiment 4.2 uses 3 seeds and Experiment 4.4 uses one run per task, both because a single fallback-ladder or RL-synthesis run can take from under a minute to (in the worst observed case for a cascading fallback through pretrained-backbone fine-tuning) over 25 minutes on our hardware. These results should be read as suggestive characterizations, not high-power statistical claims.
- Concurrent-execution contamination, caught and discarded. Our first attempt at Experiment 4.4 was run while Experiment 4.2 was still finishing on the same machine (both compete for the same local Ollama instance and CPU); that run showed an 18.75% completion rate, including LLM responses truncated mid-JSON under load. We discarded it (kept in the code repository, under a directory suffixed `_CONTENDED_DISCARDED`, for audit purposes) and re-ran Experiment 4.4 alone; the numbers reported in Table 3 are from the uncontended re-run only. We flag this because it is a concrete example of how easy it is for infrastructure-level contention to masquerade as a model-level finding if results are not cross-checked against the conditions they were collected under.
- No human-subject evaluation. We do not evaluate whether natural-language clarification dialogue elicits sufficient task specification as effectively as an ML-literate user configuring the system directly; that requires a real user study, which we did not conduct.

- Sandbox evaluation scope. Experiment 4.3 evaluates the static AST layer in isolation. We did not attempt to construct a working end-to-end exploit chaining all three defense layers, nor did we formally verify the `restricted-exec()` layer’s completeness.
- Coverage. The system also supports vision-language classification and generative vision-language fine-tuning in ongoing work not covered by this paper; only text classification and RL are evaluated here.

6 Conclusion

We presented a conversational, multi-backend system that lets an LLM generate and safely execute real training code – classification heads and full RL environments – behind a layered, empirically red-teamed sandbox and a fallback ladder that degrades gracefully when generation fails. Our experiments are deliberately modest in scale and explicit about their limitations; we believe reporting real, reproducible numbers (including failures) is more useful to the community than a stronger-sounding but unverified claim.

References

- [1] N. Erickson et al. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. arXiv:2003.06505, 2020.
- [2] M. Feurer et al. Auto-sklearn 2.0: Hands-free AutoML via Meta-Learning. JMLR, 2022.
- [3] E. Real et al. AutoML-Zero: Evolving Machine Learning Algorithms From Scratch. ICML, 2020.
- [4] Y. J. Ma et al. Eureka: Human-Level Reward Design via Coding Large Language Models. ICLR, 2024.
- [5] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.
- [6] A. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR, 2021.
- [7] M. Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032, 2024.

- [8] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.